Embedded C and CMSIS

STM32 Primer — Chapter 4

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Matters
2. What the Compiler Does
3. Why Embedded C Rather Than Assembly
4. The CMSIS Framework
5. The Three Pillars of CMSIS Register Access
6. The volatile Keyword
7. Bit Manipulation on Peripheral Registers
8. A Complete Bare-Metal Program
9. What Happens When the STM32 Starts Up
10. Summary

Why This Chapter Matters

This chapter connects two things you already know: C, and how the processor and memory work. A C variable can be a register. A C structure can be a peripheral. One line of C can set a voltage on a pin. This chapter shows how.

After this chapter, you can:

1. Describe what the compiler does with simple C: where variables live, and why arithmetic needs loads and stores.
2. Explain the CMSIS components and which one does register-level work.
3. Show how a C structure overlays a peripheral's registers.
4. Justify `volatile` with two optimizations it prevents.
5. Apply the set, clear, toggle and test idioms, and combine them into wait, read, act.
6. Write a bare-metal CMSIS program that drives a GPIO pin.

© Muním Zabidi

By the end of this chapter, you will know exactly how this one line of C changes a voltage on a physical pin:

```
GPIOA->ODR ^= (1U << 5);    // toggle the user LED on PA5
```

Questions this code raises, and where this chapter answers them:

Question raised	Answered in
What is `GPIOA->ODR`?	CMSIS: a struct laid over the registers
Why `(1U << 5)`?	Bit manipulation: building a mask
Why is nothing deleted by the compiler?	The `volatile` keyword
Why does nothing happen with the clock off?	A complete bare-metal program

Table of Contents

1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

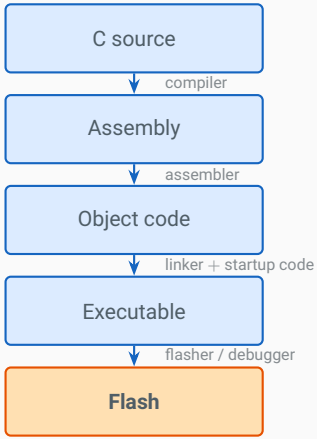
7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

What the Compiler Does



Each step is a separate program. You can inspect each one. This is how you find out what your C code actually became.

Why assembly comes next

You will write C for the rest of this book. But when something goes wrong, the debugger shows you instructions, not C.

Chapters 5–7 teach you to read that disassembly window.

The linker adds startup code. It sets the stack pointer, prepares your global variables, and calls `main()`. All of this runs before any line you wrote.

Where this leaves us

Write most code in C. Read the assembly when you need to check what the compiler did.

© Munim Zabidi

A C Statement, Compiled What the compiler writes for you

The Cortex-M4 is a load-store machine, so arithmetic happens only in registers. The compiler must move the values in and out.

```c
int x = 5;
int y = 10;
int z = x + y;
```

Notice

- Unoptimised, locals live on the stack.
- Arithmetic uses registers only.
- Optimised, the compiler drops extra memory trips, but never a register access. That needs `volatile`.

```
MOV   R0, #5          @ x = 5
STR   R0, [SP, #0]    @ store x on the stack
MOV   R1, #10         @ y = 10
STR   R1, [SP, #4]    @ store y
LDR   R0, [SP, #0]    @ reload x
LDR   R1, [SP, #4]    @ reload y
ADD   R2, R0, R1      @ z = x + y
STR   R2, [SP, #8]    @ store z
```

C is translated into instructions

The compiler decides which registers to use, which memory accesses to make, and which instructions to emit. You write what to do; the compiler writes how to do it.

© Muním Zabidi

A function is an idea in the C language. The processor has no built-in idea of a function.

When one function calls another, and later resumes, the compiler uses registers, memory, and a calling convention. The convention fixes three things:

The convention decides	On the Cortex-M4
Which registers carry the arguments	`R0`–`R3`
Where the return value is placed	`R0`
How the return address is kept	`LR`, set by `BL`

C and assembly can mix

If both follow the same convention, C can call an assembly function, and assembly can call a C function. Chapter 5 shows how.

© Munim Zabidi

Table of Contents

1 Why This Matters

2 What the Compiler Does

3 **Why Embedded C Rather Than Assembly**

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

C or Assembly?

	C	Assembly
Development speed	Fast	Slow
Readability	Good	Poor
Portability	Across ARM families	None
Code size	Good, with a modern compiler	Smallest possible
Hardware access	Through pointers and CMSIS	Direct
Typical use	Almost everything	Startup, ISR entry, hand-tuned DSP

What C gives you, and what it does not

C gives you the same hardware access as assembly, with much less bookkeeping. C does *not* give you protection. There is no runtime check to catch a bad pointer.

© Muním Zabidi

Principle	In practice
Readability	Use clear names and consistent indents. Comment on *why* a bit is set. The datasheet already says what it does.
Modularity	Small functions with one purpose. One function for each peripheral operation.
No magic numbers	Name your constants. `LED_PIN` is clearer than `(1U << 5)`.
Explicit types	Always use the fixed-width types from Chapter 3.
Defensive checks	Check ranges and returned status before you use a value.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 **The CMSIS Framework**

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

Cortex **M**icrocontroller **S**oftware **I**nterface **S**tandard. This is ARM's layer between your C code and the silicon. Every Cortex-M vendor uses the same style.

What it gives you

- A struct per peripheral, laid out on the real register offsets
- A pointer to each peripheral at its base address
- Named constants for bit fields
- Core functions: `__enable_irq()`, `NVIC_EnableIRQ()`, `__WFI()`

Why it matters

- Core parts (NVIC, SysTick) have the same names on every vendor's chip; peripherals follow the same style
- Readable: `GPIOA->ODR` rather than `*(volatile uint32_t*)0x40020014`
- The struct layout does the address arithmetic for you

You need just one header: `#include "stm32f4xx.h"`, with `STM32F446xx` defined in the project settings.

© Muním Zabidi

CMSIS is ARM's standard way to describe a Cortex-M chip in C. It has six parts:

Component	What it gives you
CMSIS-CORE	Standard access to core registers, the NVIC, SysTick and all peripheral register structures. Used throughout this book.
CMSIS-DSP	Optimised filters, FFTs, matrix and vector maths.
CMSIS-RTOS	A standard API that keeps an application independent of the real-time kernel.
CMSIS-Driver	Portable peripheral driver interfaces for middleware.
CMSIS-NN	Efficient neural-network kernels for small devices.
CMSIS-SVD	XML descriptions of every register and bit field. This is how a debugger shows registers by name.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 **The Three Pillars of CMSIS Register Access**

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

Pillar 1: A Peripheral as a Struct

```
typedef struct {
  __IO uint32_t MODER;
  __IO uint32_t OTYPER;
  __IO uint32_t OSPEEDR;
  __IO uint32_t PUPDR;
  __IO uint32_t IDR;
  __IO uint32_t ODR;
  __IO uint32_t BSRR;
  __IO uint32_t LCKR;
  __IO uint32_t AFR[2];
} GPIO_TypeDef;
```

Each member is one 32-bit register, in hardware order. C keeps the declared order, and adds no gaps between 32-bit members. So member *n* sits at offset 4*n*. These are exactly the offsets given in the reference manual.

`__IO` expands to `volatile`. CMSIS marks every register for you.

`AFR[2]` uses Chapter 3's array indexing: two real registers, reached as `AFR[0]` and `AFR[1]`.

© Muním Zabidi

Access Qualifiers: `__I`, `__O`, `__IO`

Each struct member carries a qualifier. It records what you may do with that register.

Qualifier	Access	Defined as	Example
`__I`	read-only	`volatile const`	suits `IDR`: set by hardware
`__O`	write-only	`volatile`	suits `BSRR`: a trigger
`__IO`	read/write	`volatile`	`MODER`, `ODR`

Every qualifier includes `volatile`

Writing to a `__I` member is a compile-time error. ST's header is less strict: it marks every GPIO register `__IO`, even `IDR` and `BSRR`. The next section explains why `volatile` matters.

© Muním Zabidi

The vendor defines each peripheral's base address as a macro, built up from the memory map of Chapter 2:

```
#define PERIPH_BASE      0x40000000UL                 // Peripheral region
#define AHB1PERIPH_BASE  (PERIPH_BASE + 0x00020000UL) // AHB1 bus
#define GPIOA_BASE       (AHB1PERIPH_BASE + 0x0000UL) // Port A
#define GPIOC_BASE       (AHB1PERIPH_BASE + 0x0800UL) // Port C
```

Macro	Value
GPIOA_BASE	0x40000000 + 0x00020000 + 0x0000 = 0x40020000
GPIOC_BASE	0x40000000 + 0x00020000 + 0x0800 = 0x40020800

© Muním Zabidi

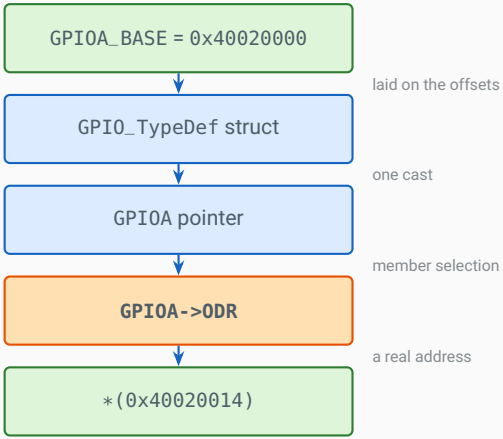
Pillar 3: From Name to Address

```
#define GPIOA_BASE  0x40020000UL

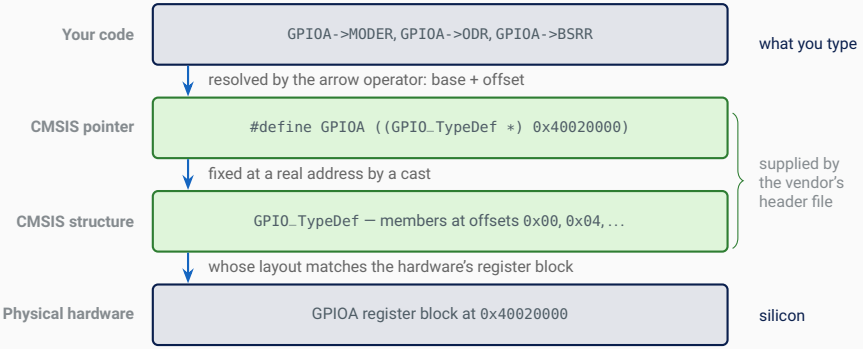
#define GPIOA \
  ((GPIO_TypeDef *) GPIOA_BASE)
```

A cast turns a plain number into a pointer to the struct. After this, the compiler computes every register's address for you.

`ODR` is the sixth member, so its offset is $5 \times 4 = 20 = 0x14$. This gives `0x40020014`, the same address as in Chapter 1.



The CMSIS Chain Hardware at the bottom, your code at the top



You only need to think about the top and bottom layers. The hardware exists. Your code names its registers. The two middle layers are a struct and a cast. CMSIS writes them once, in a header file you never edit.

A peripheral is a C structure

The structure maps C members to hardware register addresses. We can now access the registers. Next: make sure the compiler does not optimise those accesses away.

Table of Contents

1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 **The volatile Keyword**

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

`volatile` Hardware can change a value the compiler cannot see

1. Caching a value in a register

```
while ((*status & (1U << 5)) == 0)
    ;   // waits forever without
        // volatile
```

Without `volatile`, the compiler reads the address once, and never checks it again.

2. Discarding a “redundant” write

```
*reg = 1;
*reg = 0;   // without volatile the
            // first write is deleted
```

Both writes matter if the address is a pulse output.

The rule

Every pointer to a hardware register must be `volatile`. So must every variable shared with an interrupt handler.

A compiler may assume that memory only changes when the program changes it. For a status register updated by hardware, that assumption is wrong.

Next: change one bit at a time, not the whole register.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 **Bit Manipulation on Peripheral Registers**

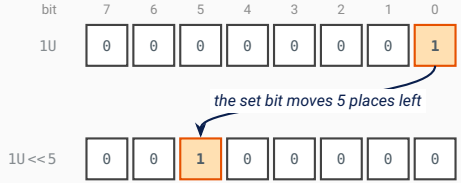
8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 Summary

Building a Mask: The Shift

Every bit idiom starts with `1U << n`. Start with `1U`: only bit 0 is set. Shift it left by *n*. Now the only set bit is bit *n*. This value is called a **mask**.



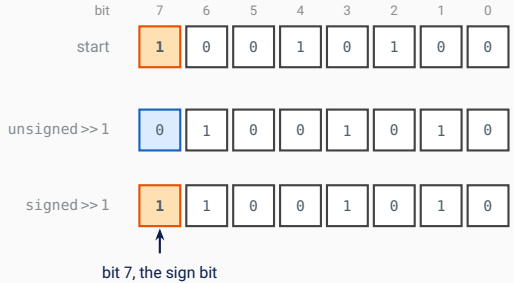
zeros fill in behind it, so exactly one bit of the mask is set.

The shift count *is* the bit number

`1U << 5` selects bit 5, not bit 6. Writing the pin number directly in the shift keeps the code readable, and easy to change for a different pin.

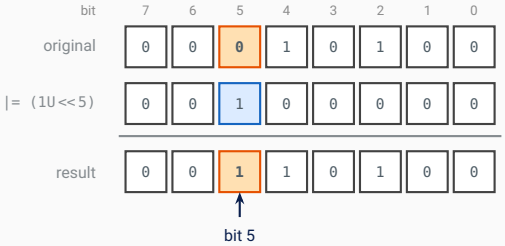
Why the U Suffix

A plain `1` is signed. Its top bit is the sign bit. C treats signed and unsigned shifts differently. This difference matters here.



`1 << 31` is undefined behaviour on a 32-bit `int`. `1U << 31` is fully defined, and gives `0x80000000`. A register is 32 switches, not a signed number. Always write `U`.

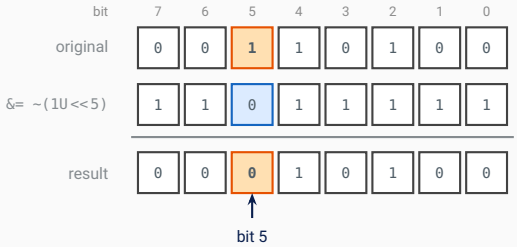
Setting a Bit: |=



OR with 1 forces a bit to 1;
OR with 0 leaves it alone.

`GPIOA->ODR |= (1U << 5);` — only the masked bit changes; every other bit is OR-ed with 0 and passes through untouched.

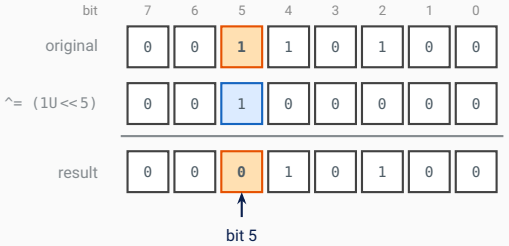
Clearing a Bit: &= with an Inverted Mask



AND with 0 forces a bit to 0;
AND with 1 leaves it alone.

`GPIOA->ODR &= ~(1U << 5);` — the tilde makes every bit except bit 5 a 1, so only bit 5 is forced low. The compiler emits `BIC`.

Toggling a Bit: ^=



XOR with 1 inverts a bit;
XOR with 0 preserves it.

`GPIOA->ODR ^= (1U << 5);` — the same statement run twice returns the register to its starting value, which is why one line suffices to blink an LED.

The Four Bit Operations

Goal	In C	Compiles to
Set bit *n*	`REG |= (1U << n);`	`ORR`
Clear bit *n*	`REG &= ~(1U << n);`	`BIC`
Toggle bit *n*	`REG ^= (1U << n);`	`EOR`
Test bit *n*	`if (REG & (1U << n))`	`TST`

Four idioms, one bit at a time

Each of these changes only the bit you name. A multi-bit field is different: the bits you want to set may already hold other values.

Multi-Bit Fields: Clear, Then Set

OR alone cannot clear a bit. Say `MODER` bits 11:10 read `11`, and you OR in `01`. The result is still `11`. You must clear the field first, before setting it.



OR alone cannot clear a bit, so a multi-bit field must be zeroed first…

…and only then written with the value you want.

`MODER` bits 11:10 currently read `11` (analog). A student writes:

```
GPIOA->MODER |= (1U << 10);   // want 01
```

Your prediction

- Which bits does the OR change?
- What do bits 11:10 hold afterwards?
- Is PA5 now a general-purpose output?

`MODER` bits 11:10 currently read `11` (analog). A student writes:

```
GPIOA->MODER |= (1U << 10);   // want 01
```

Your prediction

- Which bits does the OR change?
- What do bits 11:10 hold afterwards?
- Is PA5 now a general-purpose output?

The answer

OR can only turn bits on. `11` OR `01` is still `11`: PA5 stays analog. Clear the field first: `MODER &= ~(3U << 10);`, then OR in `01`.

A peripheral register holds settings for many pins. You must keep the bits you are not changing. `|=` and `&=` do this automatically. A plain `=` would wipe out every other setting.

A multi-bit field needs a mask: first clear it with `&= ~(3U << pos)`, then set it with `|= (value << pos)`. The mask is always "all ones for this field."

Change only the bits you intend to change

Use masks and read–modify–write operations to preserve every other setting in the register.

© Muním Zabidi

Testing Bits and Busy-Wait Polling

Test a bit

```
if ((GPIOC->IDR & (1U << 13)) == 0) {
    // PC13 is LOW:
    // the button is pressed
}
```

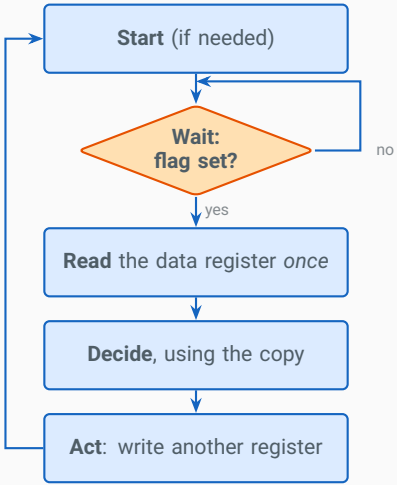
Mask the bit you want. Compare the result with zero.

Busy-wait on a flag

```
while (!(USART1->SR & (1U << 7))) {
    // wait until the transmit
    // register is empty
}
```

The loop re-reads the register each time. This works only because the register is `volatile`.

Wait, Read, Act The shape of most polled embedded code



Testing a bit says whether something happened. The busy-wait waits until it happens. Then the program reads the data and changes a different register.

- UART: wait for a received byte
- ADC: wait for a conversion
- Timer: wait for the count to finish

Very typical embedded behaviour

The registers change from one peripheral to the next. The shape of the code does not.

© Munim Zabidi

```
while (1) {
  ADC1->CR2 |= (1U << 30);               // start
  while ((ADC1->SR & (1U << 1)) == 0) {} // wait: EOC
  uint16_t v = (uint16_t)ADC1->DR;       // read once

  if (v > 2048U)                         // decide
    GPIOA->BSRR = (1U << 5);             // act: on
  else
    GPIOA->BSRR = (1U << (5 + 16));      // act: off
}
```

Input 2.0 V: SR reads `0x0`, then `0x2`. v ≈ 2482 > 2048, so BSRR gets `0x20`: LED on.

Three rules

1. Read the data once. Reading DR clears the flag.
2. Wait on the status register, not the data register.
3. Act with a safe write: BSRR for outputs.

In Chapter 9 an interrupt replaces the *wait*. Read, decide and act stay the same.

One Register Write, Viewed at Four Levels

Level	Setting one output bit
Physical hardware	The output latch and push-pull driver for pin PA5
Register	`GPIOA_ODR` bit 5, at address `0x40020014`
C	`GPIOA->ODR |= (1U << 5);`
Machine code	About four Thumb-2 encodings in Flash

One row is deliberately missing

Between the C statement and the machine code sits the assembly: the instructions the compiler really emits. The next three chapters fill in that row.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 **A Complete Bare-Metal Program**

9 What Happens When the STM32 Starts Up

10 Summary

The Clock Must Come First One rule before we write the complete program

```
RCC->AHB1ENR |= (1U << 0);
```

- Every peripheral starts with its clock disabled, to save power.
- A peripheral with no clock does not respond to writes. They are simply lost.
- `RCC` is the Reset and Clock Control peripheral. GPIOA is bit 0 of `AHB1ENR`.

The classic first bug

You write the GPIO configuration. The code compiles and runs. But nothing happens. No fault, no error: the writes went nowhere, because the clock was off.

Enable the clock first. Always.

Enable the clock before configuring a peripheral

A peripheral does not work until its clock is enabled. We now have every piece. Next: one complete program.

A Complete Bare-Metal Program Every piece, in one program

```c
#include "stm32f4xx.h"

int main(void)
{
  // 1. Enable the clock for GPIOA
  RCC->AHB1ENR |= (1U << 0);

  // 2. Set PA5 as an output
  GPIOA->MODER &= ~(3U << (5*2));
  GPIOA->MODER |=  (1U << (5*2));

  // 3. Toggle PA5 forever
  while (1) {
    GPIOA->ODR ^= (1U << 5);
    for (volatile int i = 0;
         i < 100000; i++);
  }
}
```

Every idea in this chapter

- Struct-based register access
- Bit masking and shifting
- Read–modify–write
- Clock enable, then configure

Why `5*2`? `MODER` gives each pin two bits, so pin 5's bits start at position 10.

Why `volatile` on `i`? Without it, the optimiser deletes the whole delay loop.

Step	Statement	Takeaway
1. Enable the clock	`RCC->AHB1ENR |= (1U << 0);`	The peripheral clock must be enabled first
2. Configure the pin	`MODER &= ...; MODER |= ...;`	Each GPIO pin has a two-bit configuration field
3. Control the output	`GPIOA->ODR ^= (1U << 5);`	XOR toggles the selected bit without changing the others
4. Add a delay	`for (volatile int i=0; ...)`	The compiler must not remove a delay loop whose iterations have no observable effect

Four lines, four ideas from this chapter. Every line is needed.

© Muním Zabidi

The Shape of an Embedded Program

A hosted program (on a PC)

```c
#include <stdio.h>

int main(void) {
    printf("Hello\n");
    return 0;   // back to the OS
}
```

It runs, returns, and the OS cleans up.

A bare-metal program

```c
int main(void) {
    // setup: runs once
    //   clocks, pins, peripherals

    while (1) {
        // loop: runs forever
        //   wait, read, act
    }
}
```

No OS. `main` must never return.

No OS means: no standard library (`printf` needs a screen and an OS, so output means driving a peripheral yourself), and no cleanup (the hardware stays as you leave it). *If you know Arduino:* `setup()` is the code above the loop, and `loop()` is its body.

One question left

Every example so far began inside `main()`. You never call `main()` yourself. So what calls it?

© Muním Zabidi

Table of Contents

1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 **What Happens When the STM32 Starts Up**

10 Summary



1. **Reset.** The core starts from the 16 MHz internal oscillator. This is why every example in this book uses 16 MHz.
2. **Read the vector table** at `0x08000000`. Word 0 goes into SP. Word 1 goes into PC. No software has run yet, but the CPU already has a stack and a start address.
3. **Run `Reset_Handler`.** This startup code copies initial values from Flash to SRAM, sets the rest of SRAM to zero, and calls `SystemInit()`.
4. **Call `main()`.** Now your code runs.

A global such as `int count = 5;` is ready when `main` starts, because startup code copied it there. If `main` ever returned, there would be nowhere to return to.

`main()` is not the starting point

Startup code prepares the system first, then calls `main()`.

© Muním Zabidi

Table of Contents

1 Why This Matters

2 What the Compiler Does

3 Why Embedded C Rather Than Assembly

4 The CMSIS Framework

5 The Three Pillars of CMSIS Register Access

6 The volatile Keyword

7 Bit Manipulation on Peripheral Registers

8 A Complete Bare-Metal Program

9 What Happens When the STM32 Starts Up

10 **Summary**

One C statement becomes a physical output:

```
GPIOA->ODR ^= (1U << 5);    // toggle the user LED on PA5
```

What makes it work:

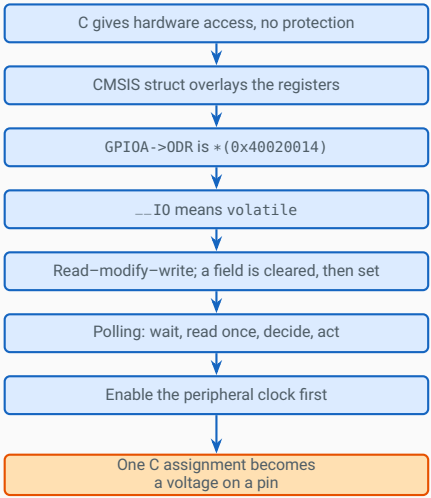
Piece	Role
Compiler	Converts C into processor instructions
CMSIS	Maps a C structure to the hardware registers
`volatile`	Preserves every access to a hardware register
Bit manipulation	Changes only the intended register bits
Clock configuration	Makes the peripheral operational

Not an abstract expression

`GPIOA->ODR ^= (1U << 5);` is a sequence of processor operations. It changes the state of a real output pin. This one line uses every idea in this chapter.

© Muním Zabidi

Summary



Terminology covered

- CMSIS, device header
- Struct overlay
- `volatile`, `__IO`
- Set, clear, toggle, test
- Wait, read, act
- Peripheral clock enable

Next: Chapter 5

ARM Assembly Fundamentals

What this C becomes, one instruction at a time.

© Munim Zabidi